



山东大学（青岛） · 计算机科学与技术学院

全国大学生计算机系统能力大赛

RespOS

操作系统内核设计文档

Rust · RISC-V 64 · LoongArch 64 · Linux ABI Compatibility

参赛队名	比特工匠队
队伍成员	李欣悦、肖安康、张俞睿
指导教师	颜廷坤、潘润宇
完成日期	2026 年 6 月

初赛设计文档

目 录

1 概述	5
1.1 项目介绍	5
1.2 代码结构	5
1.3 设计原则	6
1.4 整体架构	6
1.5 团队分工	7
1.6 初赛进展与排名 TODO	8
2 进程管理	9
2.1 概述	9
2.2 任务调度	9
2.3 任务调度队列与执行器	10
2.4 任务控制块	11
2.4.1 进程和线程联系	12
2.4.2 任务的状态	12
3 中断与异常处理	14
3.1 特权级切换	14
3.2 处理过程	15
3.2.1 内核态异常处理	15
3.2.2 用户态中断与异常处理	15
3.2.3 返回用户态	16
4 内存管理	17
4.1 地址空间总览	17
4.1.1 地址空间布局	17
4.1.2 地址翻译模式	18
4.1.3 Boot 阶段的预映射	19
4.1.4 内核地址转换	19
4.2 物理内存管理	19
4.2.1 物理页帧分配器	19
4.2.2 物理页帧的生命周期管理	19
4.3 进程内存管理	20
4.3.1 虚拟内存区域	20
4.3.2 进程地址空间	20
4.4 缺页异常处理	20
4.4.1 进程缺页异常概述	20
4.4.2 写时复制机制	21
4.4.3 懒分配机制	21
4.5 内核动态内存分配	22
4.6 用户地址检查	22

5 文件系统	23
5.1 虚拟文件系统	23
5.1.1 SuperBlock	23
5.1.2 Inode	24
5.1.3 Dentry	24
5.1.4 File	24
5.2 磁盘文件系统	24
5.2.1 EXT4 文件系统	24
5.3 非磁盘文件系统	25
5.3.1 procfs	25
5.3.2 devfs	25
5.4 页缓存	25
5.5 文件描述符表	26
6 进程间通信	27
6.1 信号机制	27
6.2 信号传输	27
6.3 信号处理	28
6.4 管道 (Pipe)	28
6.4.1 Pipe 设计	28
6.4.2 读端与写端通信	29
6.5 System V IPC 机制	29
6.5.1 System V IPC 对象	30
6.5.2 IPC Key 管理器	30
6.5.3 System V 共享内存	30
6.6 Futex 同步	30
7 时钟模块	31
7.1 定时器队列	31
7.1.1 时间轮设计	31
7.2 定时器	31
8 网络模块	32
8.1 Socket 套接字	32
8.2 Ethernet 设备	32
8.3 传输层: UDP 与 TCP	32
8.4 Port 端口分配	32
9 设备	33
9.1 设备管理模块概述	33
9.2 设备树	33
10 硬件抽象层	34
10.1 硬件抽象层总览	34
10.2 处理器访问接口	34

10.3 内核入口例程	34
10.4 内存管理单元与地址空间	34
10.4.1 物理内存	34
10.4.2 分页地址翻译模式	34
10.4.3 页表	34
10.4.4 直接映射窗口	34
10.4.5 TLB 重填	34
11 总结与展望	35
11.1 工作总结	35
11.2 未来计划	35
11.3 参考	35
12 AI 协作与开源项目借鉴	36
12.1 AI 辅助完成的工作	36
12.2 人工审核与边界控制	36
12.3 借鉴的开源项目	36
12.4 许可证与引用说明	36

第一章

概述

1.1 项目介绍

RespOS 是一个使用 Rust 语言编写的类 Unix 宏内核操作系统，面向操作系统能力大赛初赛测例与教学型内核演进场景。项目当前以 Linux ABI 兼容为主要目标，围绕进程管理、虚拟内存、文件系统、信号、IPC、时间、网络和设备驱动等基础能力展开实现，使 `busybox`、`libc` 初始化流程和 LTP 测例能够在 QEMU 虚拟平台中运行。

在硬件平台方面，RespOS 同时适配 RISC-V 64 与 LoongArch 64。两套架构在启动入口、页表机制、异常上下文和返回用户态流程上存在差异，因此项目将架构相关代码集中收敛到 `arch` 模块中，由上层任务、内存、文件系统和系统调用模块通过统一接口使用底层能力。这样的设计可以减少上层模块对具体指令集的依赖，也方便后续继续补齐双架构行为一致性。

初赛阶段，RespOS 的实现重心不是单独堆叠系统调用入口，而是补齐用户程序运行链路中的关键内核基础设施。例如 `execve` 需要路径解析、ELF 装载、地址空间重建、用户栈构造和辅助向量；`fork/clone` 需要处理任务控制块、地址空间、文件描述符表和信号状态；文件相关测例则依赖 VFS、Ext4 后端、dentry cache、page cache 和 Linux 风格错误码共同工作。

目前项目仍处于持续开发阶段。现有内核已经形成了比较完整的模块边界，能够支撑后续围绕测例结果继续补齐细节语义、修复兼容性缺口和优化稳定性。

模块	当前设计重点
架构层	维护 RISC-V 64 与 LoongArch 64 的启动、trap、页表、上下文切换和平台接口。
任务管理	提供任务控制块、调度队列、内核栈、父子关系、 <code>fork/exec/wait</code> 和 <code>futex</code> 等机制。
内存管理	实现页帧分配、内核高地址映射、用户地址空间、ELF 装载、 <code>mmap</code> 和用户指针访问。
文件系统	构建 VFS、路径解析、文件描述符表、 <code>pipe</code> 、 <code>procfs/devfs</code> 、Ext4 后端和缓存层。
兼容接口	围绕 Linux ABI 实现系统调用分发、 <code>errno</code> 、信号、时间、IPC、网络回环和系统信息接口。

表 1 初赛阶段核心模块概览

1.2 代码结构

RespOS 仓库按照内核、用户程序、镜像脚本、测试工具和第三方依赖划分目录。内核主体位于 `os/`，用户态测试程序位于 `user/`，`vendor/` 中保存当前阶段需要直接纳入构建的第三方代码，`judge/` 与 `scripts/` 用于测例统计、镜像处理和辅助构建。

```
RespOS/
├─ Makefile           # 顶层构建入口
├─ bootloader/        # RustSBI 等启动固件
├─ docs/              # 开发记录、模块说明和本文档
```

```

├─ img/                # RISC-V / LoongArch 测试镜像
├─ judge/              # LTP 日志过滤、对比和报告脚本
├─ scripts/            # 镜像获取、测试统计等辅助脚本
├─ testsuit/           # 比赛测例相关材料
├─ user/               # 用户态程序、shell 和简单功能测试
├─ vendor/             # lwext4_rust、smoltcp、riscv 等依赖
├─ os/                 # RespOS 内核源码
│   ├── Cargo.toml
│   ├── build.rs
│   └── src/
│       ├── arch/       # RISC-V 64 / LoongArch 64 架构相关代码
│       ├── drivers/    # VirtIO block 等设备驱动
│       ├── fs/         # VFS、Ext4、procfs、devfs、pipe、缓存
│       ├── mm/         # 页帧、堆、页表和地址空间管理
│       ├── net/        # socket、UDP/TCP、本地回环通信
│       ├── signal/     # 信号处理、sigaction、sigreturn 相关结构
│       ├── syscall/    # Linux ABI 系统调用分发与参数转换
│       ├── task/       # 任务控制块、调度器、内核栈、futex
│       ├── mutex/      # 自旋锁、睡眠锁和同步封装
│       ├── loader.rs   # 用户程序装载辅助
│       └── main.rs     # 内核 Rust 入口与初始化流程

```

内核初始化入口在 `os/src/main.rs`。早期入口完成 BSS 清零和架构必要准备后进入 `rust_main_high`，随后依次初始化 trap、内存、网络、首个用户进程、定时器中断，最后进入任务调度循环。这个顺序反映了当前系统的依赖关系：任务创建依赖内存管理，用户程序运行依赖 trap 和地址空间，文件系统、网络 and 系统调用能力则在用户态负载执行过程中被逐步触发。

1.3 设计原则

RespOS 的实现围绕几个贯穿全文的原则展开。第一，采用宏内核结构，让任务、内存、文件系统和信号模块可以在一次系统调用内直接协作，优先降低初赛阶段补齐 Linux ABI 的工程成本。第二，使用 Rust 所有权和 RAII 管理资源生命周期：页帧由 `FrameTracker` 回收，任务、inode、dentry、FileOp 和共享内存页通过 `Arc/Weak` 表达共享关系。第三，保持调度策略简单，把复杂度留给阻塞、唤醒、退出和信号中断边界；当前 FIFO 队列不追求策略复杂度，而追求状态转换可解释。第四，用户地址永远不在内核中直接解引用，而是先检查 VMA 权限并主动处理懒分配或 COW，避免把普通系统调用错误退化成不可控的内核缺页。

这些原则不是抽象口号，而是直接影响后续章节的结构：第二章的 TCB 聚合任务资源，第三章把 trap 返回作为信号递送点，第四章把地址合法性和页分配状态拆开，第五章用 VFS trait object 统一文件对象，第六章则让管道、共享内存和 futex 分别落在 fd、页帧和用户地址三个边界上。

1.4 整体架构

RespOS 采用宏内核结构，各核心模块运行在同一内核地址空间中。初赛阶段选择这一结构，主要是为了降低跨模块协作成本：`execve`、`fork`、`mmap`、`pipe`、`signal` 和路径解析等接口都不是单模块功能，需要任务、内存、文件系统、信号和系统调用层共同维护 Linux ABI 语义。

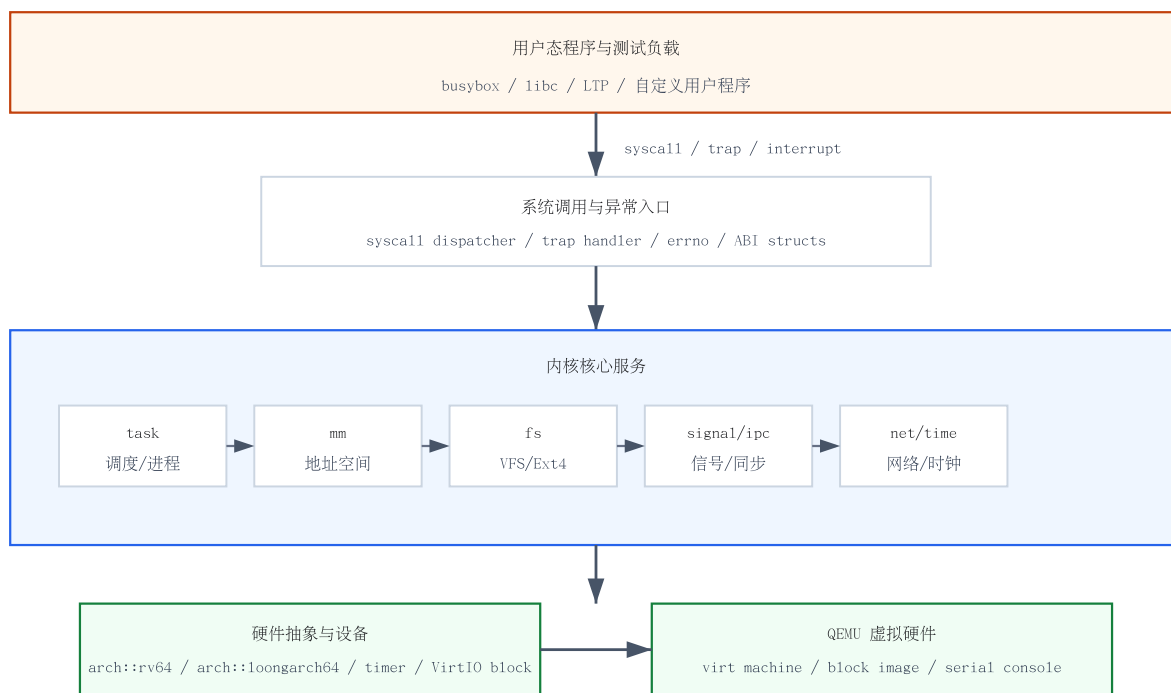


图 1 RespOS 整体架构示意

这种结构的优点是开发路径直接，便于在测例驱动下快速补齐能力；代价是模块之间存在真实的语义耦合。因此项目需要通过接口约定和回归测试约束资源生命周期，避免文件描述符、地址空间、信号状态和父子关系在复杂系统调用路径中出现不一致。

以用户执行 `cat /proc/cpuinfo` 为例，shell 先通过 `fork` 创建子任务，子任务执行 `execve` 装载 `cat`；系统调用层进入文件系统后，VFS 从当前 `cwd/root` 出发解析 `/proc/cpuinfo`，挂载树把路径切换到 `procfs` 后端；`procfs` 的 `inode` 动态生成 CPU 信息文本，常规 `read` 再通过 `fd` 表中的 `FileOp` 返回给用户缓冲区。这个短路径同时穿过任务、`trap`、用户地址检查、VFS、`procfs` 和 `fd` 表，正是 RespOS 选择宏内核和清晰模块边界的原因。

1.5 团队分工

当前项目由三名队员协作推进，张俞睿担任队长并承担项目主要开发工作。由于内核模块之间耦合较高，实际开发中会根据测例压力和 bug 所在模块动态交叉支持；下表描述的是初赛文档阶段的主要责任边界，后续会随开发进度继续更新。

成员	主要工作
张俞睿（队长）	负责项目总体设计和主要代码实现，推进内核主体开发、双架构适配、任务/内存/文件系统/系统调用等核心模块联调，并维护构建脚本、测例辅助工具和阶段性问题定位。
李欣悦	负责文档统筹、测试记录整理、系统调用兼容性梳理以及部分模块联调，协助维护初赛测例运行流程。
肖安康	参与内核功能开发与调试，协助任务管理、内存管理、异常处理、文件系统与 Linux ABI 兼容相关实现。

表 2 团队分工

在协作方式上，团队优先围绕“能否运行用户态负载”和“测例失败原因是否可定位”推进工作。对于影响多个模块的功能，会先明确系统调用语义、涉及的数据结构和错误码边界，再进入具体实现；对于不稳定测例，会保留日志、复现步骤和阶段性处理方案，避免临时修补分散在难以审查的位置。

1.6 初赛进展与排名 TODO

截至本文档当前版本，RespOS 仍处于初赛功能补齐和稳定性改进阶段。项目已经建立双架构源码结构，并围绕文件系统、任务管理、内存管理、信号、IPC、网络回环和设备驱动形成了可继续迭代的内核主体。后续将继续以初赛测例结果为依据，补齐 Linux ABI 的边界行为和错误路径。

TODO：初赛排名图

当前比赛仍在开发和测试过程中，最终排名截图、测例通过情况和阶段性结果图将在初赛结果稳定后补充到本节。

本章小结

概述章给出 RespOS 的目标、代码组织和团队责任边界。后续章节将沿着用户程序运行所依赖的主路径展开：任务被调度，`trap` 进入内核，地址空间提供隔离，文件系统和其他内核服务再向 Linux ABI 补齐具体语义。

第二章

进程管理

2.1 概述

进程管理模块负责把用户程序抽象为可调度、可等待、可复制、可替换的内核对象。RespOS 当前将“任务”作为调度基本单位，用 `TaskControlBlock` 描述一个线程级执行实体；多个任务可以通过 `ThreadGroup` 组成一个 Linux 风格线程组，此时线程组 ID (TGID) 对应传统意义上的进程 ID，而每个任务仍拥有独立的线程 ID (TID)。

从实现角度看，进程管理并不是孤立模块。创建任务需要内存模块提供地址空间和内核栈映射；执行 `execve` 需要文件系统提供可执行文件数据，并由内存模块重建用户地址空间；`wait4` 需要维护父子关系和退出状态；线程退出还会与 `futex`、信号和 `clear_child_tid` 等用户态同步机制协作。因此本章重点描述 RespOS 如何组织任务对象、如何调度任务，以及如何在 Linux ABI 语义下区分进程和线程。

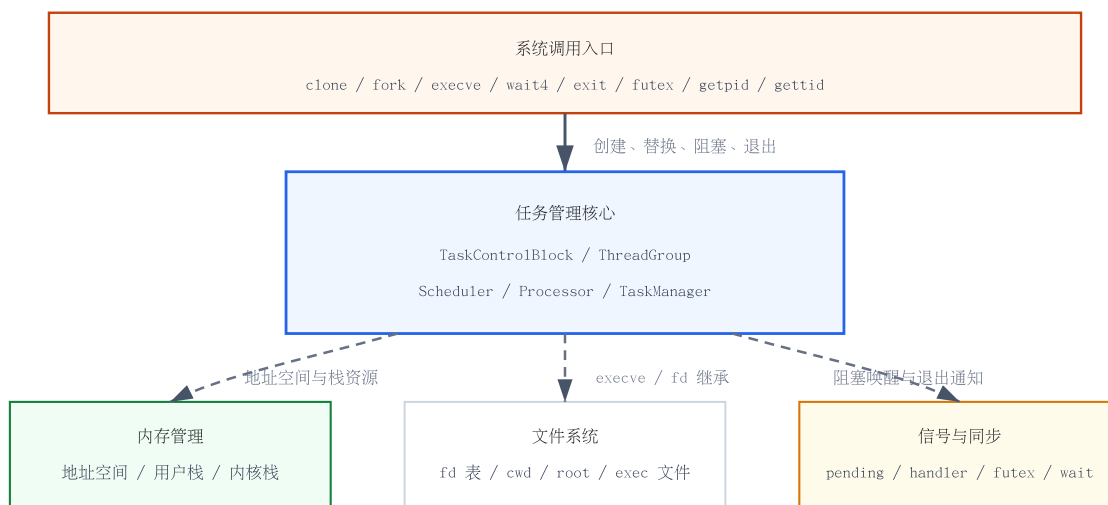


图 2 进程管理模块在内核中的位置

进程管理模块的主要源文件位于 `os/src/task/`。其中 `task.rs` 定义任务控制块和进程/线程资源，`scheduler.rs` 维护就绪队列与阻塞队列，`processor.rs` 记录当前 CPU 正在运行的任务，`manager.rs` 提供 TID 到任务对象的全局弱引用索引，`context.rs` 和 `kstack.rs` 则分别负责任务上下文和内核栈布局。

2.2 任务调度

RespOS 当前采用简单直接的 FIFO 调度策略。就绪任务被放入 `ready_queue`，调度器每次从队首取出下一个任务执行；当任务主动让出 CPU、阻塞等待事件、收到停止信号或退出时，调度路径会更

新任务状态，并通过架构层 `__switch` 切换到下一个任务的内核栈和地址空间。`__switch` 只负责内核上下文切换，真正返回用户态还要依赖第三章描述的 `__restore`。

任务首次运行时，内核在任务内核栈上放置 `TrapContext` 和 `TaskContext`。`TaskContext` 的返回地址指向 `__restore`，因此任务被调度后会进入异常返回流程，再恢复到用户态入口。后续任务切换时，`__switch` 保存当前任务的内核上下文，恢复下一个任务的内核栈指针、返回地址和页表 `token`，使任务能够从之前暂停的位置继续执行。

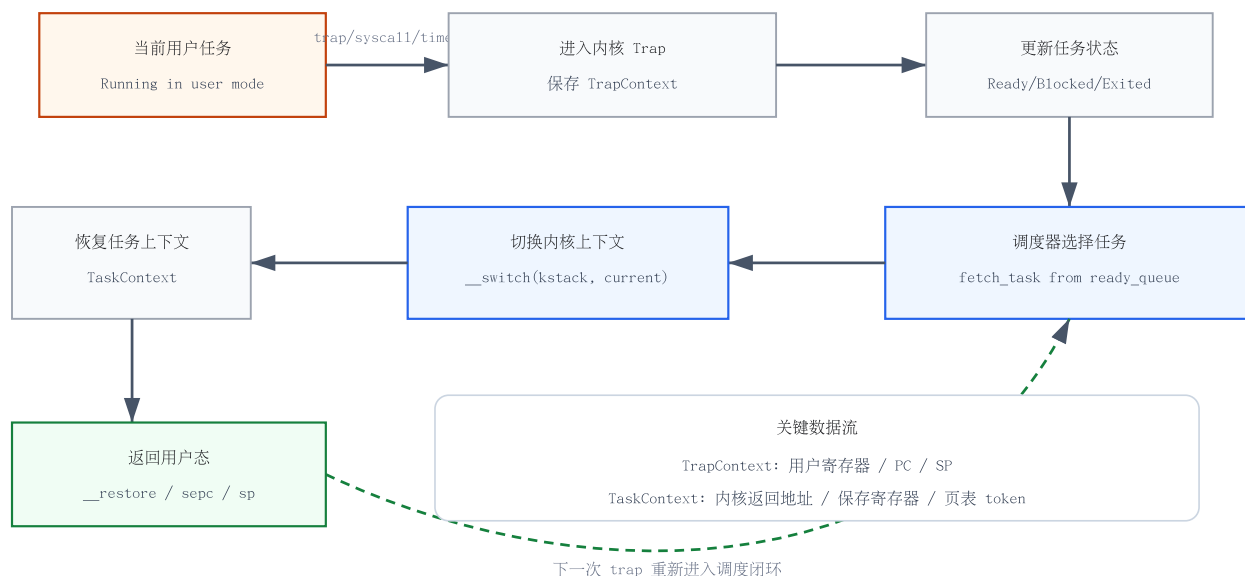


图 3 任务切换流程

调度器目前不引入复杂优先级、时间片权重或多核心负载均衡，主要目标是保证初赛阶段系统调用和用户态程序运行链路稳定。对于 `busybox`、`libc` 初始化和 LTP 测例而言，调度策略本身不是瓶颈，关键在于阻塞、唤醒、退出和父子回收路径必须语义清楚，不能丢失任务状态或错误释放资源。

2.3 任务调度队列与执行器

调度器由两个核心队列组成：`ready_queue` 保存可运行任务，`blocked_queue` 保存因等待事件而暂时不可运行的任务。`add_task` 会将就绪任务加入队尾，`fetch_task` 从队首取出任务，`block_task` 将阻塞任务放入阻塞队列，`wakeup_task` 则根据 TID 将任务从阻塞队列移回就绪队列。

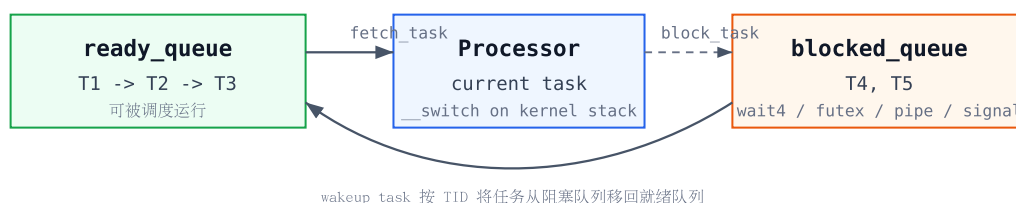


图 4 调度队列结构

`processor.rs` 中的 `PROCESSOR` 保存当前 CPU 正在运行的任务。系统启动后，`run_tasks` 从就绪队列取出第一个任务，将其记录为当前任务，再切换到该任务内核栈。此后常规调度不再回到一个复杂的执行器对象，而是由当前任务在让出、阻塞或退出路径中直接选择下一个任务并调用 `__switch`。

为了便于按 TID 查找任务，TaskManager 维护一个 `HashMap<tid, Weak<TaskControlBlock>>`。调度队列负责“谁接下来运行”，任务管理器负责“能否通过 TID 找到任务对象”，二者职责不同。这样的拆分让 `kill`、`futex wake`、`wait` 等路径可以按 TID 找到目标任务，而不会强依赖任务是否仍在 `ready` 队列中。

结构	职责
Scheduler	维护 FIFO 就绪队列和阻塞队列，负责添加、取出、阻塞、唤醒和移除任务。
Processor	记录当前 CPU 正在运行的任务，为 <code>current_task</code> 和上下文切换提供当前任务引用。
TaskManager	维护 TID 到任务对象的全局弱引用索引，支持按 TID 查询和遍历任务。
KernelStack	为每个任务分配内核栈 <code>slot</code> ，并在任务生命周期内维护栈顶位置。
TaskContext	保存任务切换所需的返回地址、被调用者保存寄存器和页表 <code>token</code> 。

表 3 调度相关结构职责

2.4 任务控制块

TaskControlBlock 是任务管理模块最核心的数据结构，用于把一个可调度实体的身份、运行上下文和进程资源聚合到同一个生命周期中。为了支持中断、系统调用和唤醒路径中的并发访问，内部可变字段通常通过 `SpinLock`、`RwLock` 或原子变量保护；具体字段按职责分组如下。

身份与关系	运行上下文	进程资源
<code>tid / tgid / pgid / sid</code>	<code>kernel_stack / task_status</code>	<code>fd_table / cwd / root</code>
<code>parent / children / exit_code</code>	<code>memory_set / TaskContext</code>	<code>exe_path / limits</code>
信号资源	线程同步	计时统计
<code>sig_pending / sig_stack</code>	<code>clear_child_tid</code>	<code>itimers / start_time</code>
<code>sig_handler</code>	<code>robust_list / futex</code>	<code>child_utime / child_stime</code>

表 4 TaskControlBlock 结构分组

任务创建时会先分配 TID 和内核栈，再构造用户地址空间，并在内核栈顶部布置 `TrapContext` 与 `TaskContext`。`execve` 不创建新的任务对象，而是在当前任务上替换地址空间、重建用户栈、关闭 `close-on-exec` 文件描述符，并重置信号处理状态。

```
pub struct TaskControlBlock {
    kernel_stack: KernelStack,
    tid: RwLock<TidHandle>,
    memory_set: Arc<RwLock<MemorySet>>,
    fd_table: SpinLock<Arc<FdTable>>,
    sig_pending: SpinLock<SigPending>,
}
```

2.4.1 进程和线程联系

RespOS 采用接近 Linux 的“线程是调度单位，进程是线程组”的模型。每个 `TaskControlBlock` 都是一个可调度任务，拥有唯一 TID；当多个任务属于同一个 `ThreadGroup` 时，它们共享 TGID，用户态通常把 TGID 视为进程 ID。`gettid` 返回当前任务的 TID，`getpid` 返回当前线程组的 TGID。

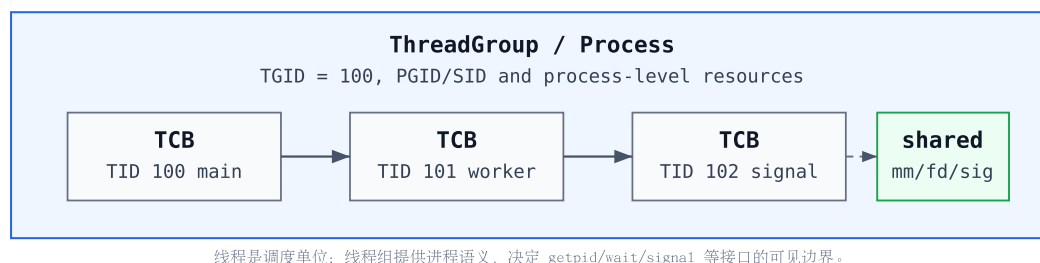


图 5 进程与线程关系

`clone` 标志决定新任务是线程还是进程。当设置 `CLONE_THREAD` 时，新任务加入调用者的线程组，并共享地址空间、部分文件系统上下文和信号处理函数表；未设置该标志时，新任务成为独立进程，拥有新的 TGID，并作为子进程加入父任务的 `children` 表。这个区分直接影响 `wait4` 回收语义：普通子进程由父进程等待回收，同线程组内的新线程不进入父进程的 `children` 表。

这种模型使 RespOS 能够兼容 `libc` 和 `pthread` 中常见的线程创建方式，同时保留传统 `fork/exec/wait` 进程语义。对于初赛测例而言，正确处理 TID、TGID、父子关系和线程组退出，是 `shell`、`busybox` 和 `LTP` 进程类测例能否稳定运行的关键。

2.4.2 任务的状态

任务状态由 `TaskStatus` 表示，它不只是调度器内部标记，也会影响 `wait4`、信号处理、`futex` 唤醒和资源释放。状态变化必须与队列操作保持一致，否则容易出现任务已经阻塞但仍被调度、任务已经退出但无法被父进程回收等错误。

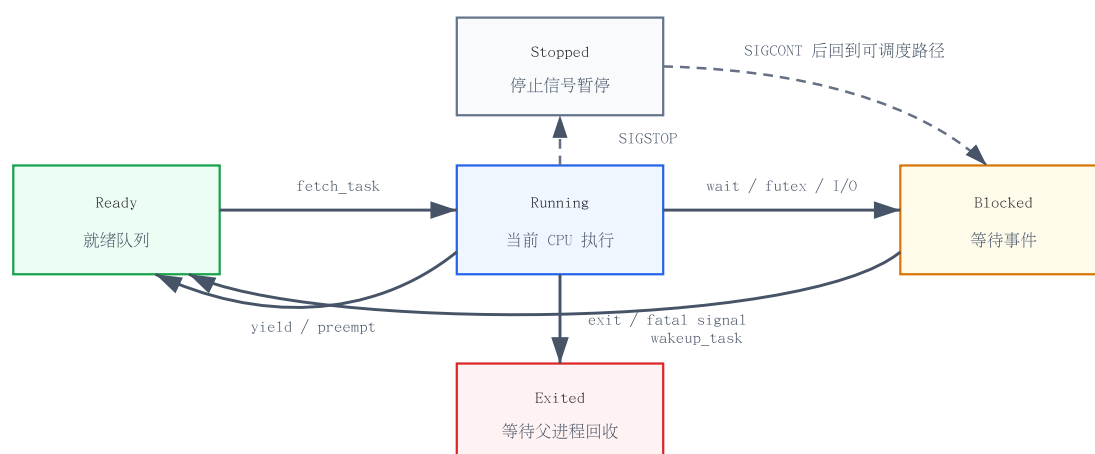


图 6 任务状态转换

其中 `Exited` 不是立即释放所有资源的终点。普通子进程退出后仍需要保留退出码和必要的任务元数据，直到父进程通过 `wait4` 完成观察和回收；线程退出还需要配合 `futex`、`clear_child_tid` 和线程组状态完成用户态同步。

本章小结

进程管理章的关键设计是把调度单位、线程组语义和进程资源生命周期分开处理。这样 `clone`、`fork`、`execve`、`wait4`、信号和 `futex` 可以共享同一套任务对象，同时仍能保留 Linux 对 TID、TGID、父子回收和文件偏移共享的预期。

第三章

中断与异常处理

3.1 特权级切换

中断与异常处理模块负责把用户态执行流安全转入内核，并在处理完成后恢复用户态上下文。RespOS 在 RISC-V 64 和 LoongArch 64 上分别实现架构相关入口，但上层处理抽象保持一致。整体流程是入口汇编保存 `TrapContext`，Rust 侧 `trap_handler` 完成异常分发，最后通过 `__restore` 返回用户态或进入调度路径。

从用户态进入内核时，硬件只保存最小异常现场并跳转到架构指定入口。两套架构都不会自动保存所有通用寄存器，因此 RespOS 在入口汇编中显式构造完整 `TrapContext`。

```
__trap_from_user:
    csrrw sp, sscratch, sp
    addi sp, sp, -68*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    csrr t0, sstatus
    csrr t1, sepc
```

项目	RISC-V 64	LoongArch 64
异常入口	stvec 指向 <code>__trap_from_user</code> / <code>__trap_from_kernel</code>	EENTRY 指向用户或内核 trap 入口
返回指令	sret	ertn
返回地址	sepc	ERA
异常原因	scause	ESTAT
坏地址	stval	BADV
用户/内核栈交换	sscratch 保存用户栈和内核栈切换信息	KSAVE0 保存用户栈和下次入口内核栈顶

表 5 双架构 trap 关键信息对照

`TrapContext` 的设计承担两个职责：一是保存用户态被打断时的通用寄存器、返回地址和特权状态；二是作为系统调用参数和返回值的传递载体。RISC-V 使用 `a7/x17` 作为系统调用号、`a0-a5/x10-x15` 作为参数并把返回值写回 `a0`；LoongArch 使用 `a7/r11` 作为系统调用号、`a0-a5/r4-r9` 作为参数并把返回值写回 `a0/r4`。因此系统调用分发层可以使用统一的 `syscall(id, args)` 接口，而寄存器细节被收敛在各架构 `TrapContext` 实现中。

3.2 处理过程

RespOS 的 trap 处理分为入口保存、Rust 分发、后置检查和返回恢复四个阶段。入口汇编只做与架构强相关的工作，包括保存寄存器、切换异常入口、准备 `&mut TrapContext` 参数；具体语义由 Rust 侧完成，这样系统调用、缺页、信号和定时器逻辑可以尽量复用上层模块。

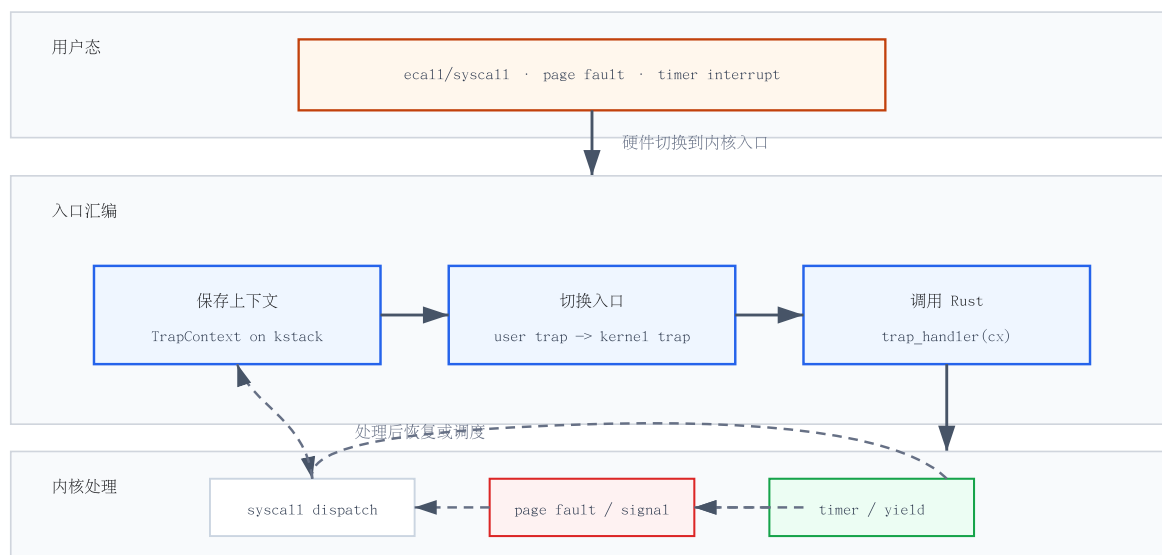


图 7 用户态 trap 处理流程

图中各分支的处理结果并不相同：有的修改上下文后继续返回，有的先尝试修复，失败后转为信号或退出，有的直接进入调度。具体边界在用户态 trap 表中列出。

3.2.1 内核态异常处理

内核态异常和用户态异常采用不同入口。用户态进入 trap 后，入口汇编会把后续异常入口切到内核 trap 路径；返回用户态前再切回用户 trap 路径。这样如果内核在处理系统调用或缺页时再次触发异常，不会错误地按用户态现场保存方式处理。

当前内核态 trap 的策略偏保守：断点异常可以打印信息并跳过断点指令；非法指令、内核缺页、内核态系统调用等情况直接 panic。原因是这些异常通常代表内核 bug 或关键假设被破坏，继续执行可能扩大破坏范围。定时器中断若出现在内核态，目前主要记录日志，不在内核路径中执行复杂调度。

3.2.2 用户态中断与异常处理

用户态 trap 是系统调用、缺页、非法指令、断点和定时器中断的统一入口。对于可恢复事件，内核会修改 `TrapContext` 后返回用户态；对于不可恢复事件，内核会向任务递送信号或直接终止任务。不同 trap 类型的修复动作和最终状态如下。

trap 类型	处理模块	修复动作	最终状态
系统调用	syscall	跳过 syscall 指令，分发到具体实现，写回返回值或错误码。	返回用户态; sigreturn 可直接恢复旧上下文。
缺页异常	mm::MemorySet	根据访问类型尝试懒分配、写时复制、权限检查和 VMA 修复。	修复成功后返回；失败时递送 SIGSEGV 或结束任务。
定时器中断	timer / task	设置下一次触发，检查任务定时器和 POSIX timer。	当前任务让出 CPU，进入调度路径。
非法指令	task	记录异常现场。	结束当前任务。
断点异常	task	记录断点位置。	结束当前任务。

表 6 用户态 trap 类型与处理结果

在用户态 trap 返回前，内核还会统一检查任务定时器和待处理信号。这样做的好处是信号递送点比较集中：系统调用返回、缺页修复后返回、定时器触发后的调度出口都可以进入同一套 `handle_signals` 逻辑。对于 libc、pthread 和 LTP 测例来说，信号能否在阻塞等待、定时器到期和系统调用返回边界被及时观察，是兼容性的重要部分。

3.2.3 返回用户态

返回用户态由 `__restore` 完成。该路径从当前任务内核栈上的 `TrapContext` 恢复通用寄存器、返回地址和特权状态，并将异常入口重新设置为用户 trap 保存路径。RISC-V 在恢复 `sstatus` 与 `sepc` 后执行 `sret`；LoongArch 在恢复 `PRMD` 与 `ERA` 后执行 `ertrn`。从用户程序视角看，除了系统调用返回值、信号处理上下文或异常导致的退出外，普通 trap 应当像一次透明的控制流暂停。

返回路径还承担下一次 trap 的准备工作。RISC-V 使用 `sscratch` 保存用户栈和内核栈切换所需的信息；LoongArch 在返回前把内核栈顶写入 `KSAVE0`，并把内核 `tp` 保存到 `KSAVE1`，保证下一次用户态异常进入内核后可以恢复正确的内核线程局部状态。

`__restore` 也是调度路径和 trap 路径的交汇点。首次运行任务时，调度器恢复的 `TaskContext` 会跳到 `__restore`，由它把预先布置在内核栈上的 `TrapContext` 恢复成用户态入口；普通系统调用返回时，`trap_handler` 修改同一份 `TrapContext` 后也回到 `__restore`。因此这里必须同时保证当前返回正确、下一次异常入口正确、架构特权状态正确，任何一个细节出错都会表现为用户栈损坏、重复执行系统调用或无法从信号处理返回。

本章小结

中断与异常处理章的重点是把双架构入口差异收敛到统一的 `TrapContext` 和 `Rust` 分发逻辑中。系统调用、缺页、定时器和信号递送最终都通过 `trap` 返回路径闭合，因此 `__restore` 同时是异常处理、调度和用户态恢复的共同边界。

第四章

内存管理

4.1 地址空间总览

内存管理模块负责维护内核和用户进程的虚拟地址空间，为任务创建、`execve`、`fork` 等十余个路径提供基础能力。RespOS 将地址空间抽象为 `MemorySet`，其中包含根页表、逻辑段列表、堆边界和 `mmap` 分配游标；每个逻辑段由 `MapArea` 表示，描述一段连续虚拟页的映射类型、访问权限和已经分配的物理页帧。

在双架构支持上，RISC-V 64 和 LoongArch 64 都采用 4 KiB 页和 39 位虚拟地址宽度，内核高地址线性映射、用户低地址空间、`mmap` 区间和 `sigreturn` 跳板页保持同一套常量布局。架构差异集中在页表项编码、页表根写入方式和 TLB 刷新指令上，上层 `MemorySet` 通过统一的 `PageTable`、`PageTableEntry` 和 `MapPermission` 接口使用这些能力。

项目	RISC-V 64	LoongArch 64
页表 token	<code>satp = MODE(Sv39) root_ppn</code>	<code>root_ppn << 12</code> ，写入页表根 CSR
页表根寄存器	<code>satp</code>	<code>PGDL</code> 和 <code>PGDH</code> 同时写入当前 <code>root</code>
TLB 刷新	<code>sfence.vma</code>	<code>invtlb</code> 刷新全局和 ASID 相关项
PTE 软件位	使用 <code>RSW</code> 位保存 <code>COW</code> 标志	使用软件位保存 <code>COW</code> 标志，并转换为 LoongArch PTE 编码

表 7 双架构页表机制对照

4.1.1 地址空间布局

RespOS 采用“用户低地址、内核高地址”的布局。用户进程的页表并不是从零开始构造完整内核映射，而是通过 `PageTable::from_kernel` 复制内核高地址部分的根页表项，使每个用户地址空间都能在陷入内核后继续访问内核代码、数据、堆、设备映射和物理页线性映射。

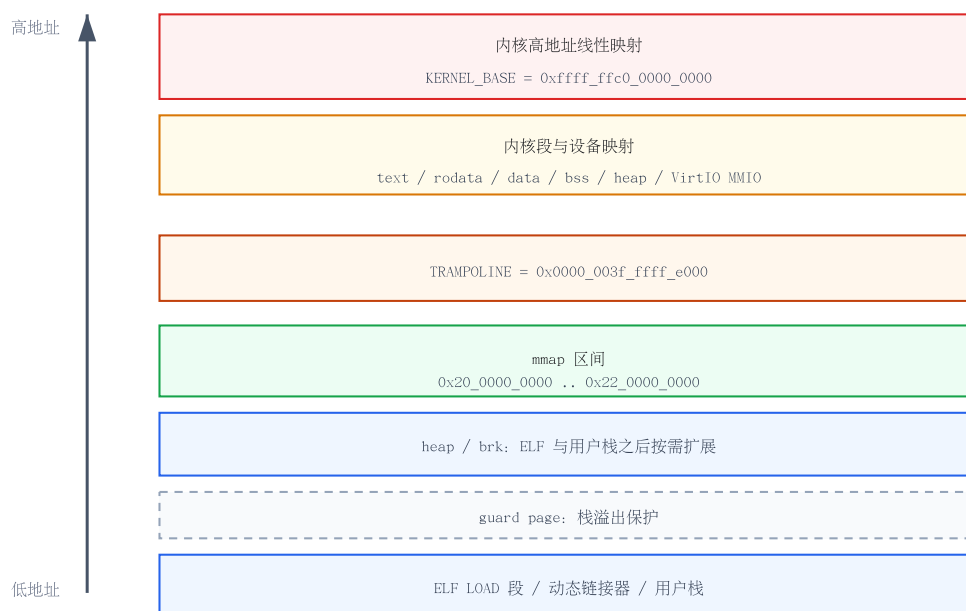


图 8 RespOS 地址空间布局

用户地址空间由 ELF LOAD 段、动态链接器映射、用户栈、堆区、mmap 区间和 sigreturn 跳板页组成。加载 ELF 时，内核根据程序头权限建立 **READ**、**WRITE**、**EXECUTE**、**USER** 标志；最高 LOAD 段之后先留出一页 guard page，再放置用户栈，栈顶位于栈区域高地址端，向低地址增长时会先撞到 guard page。heap_bottom 和初始 brk 放在用户栈之后，堆通过 brk 向高地址扩展；mmap 区间从 MMAP_MIN_ADDR 独立开始，和早期堆/栈之间保留较大空洞，避免普通堆增长立即碰到 mmap 区。

4.1.2 地址翻译模式

地址翻译先把虚拟地址按 4 KiB 页大小拆成 VPN 和页内偏移，再用 VPN 的三级索引沿根页表逐级查找页表项。中间级页表项不存在时，find_pte_create 会分配新的页表页；叶子页表项有效时，内核取出 PPN，并把 PPN 对应物理页基址加上原始页内偏移，得到最终物理地址。

VirtAddr、VirtPageNum、PhysAddr 和 PhysPageNum 负责地址与页号转换；PageTable 负责沿三级页表查找或创建页表项；MapArea 决定虚拟页最终映射到哪类物理页。内核直接映射区使用 MapType::Direct，物理页号可以由虚拟页号减去 KERNEL_BASE 对应页号得到；用户空间和 mmap 区域使用 MapType::Framed，由页帧分配器按需分配物理页。

结构	职责
MemorySet	维护一个地址空间的根页表、逻辑段、堆边界和 mmap 游标。
MapArea	描述连续虚拟页区间的映射类型、权限、共享属性和已分配页帧。
PageTable	创建三级页表、建立/取消映射、修改 PTE 标志并生成架构 token。
FrameTracker	持有一个物理页帧，离开生命周期后自动归还页帧分配器。

表 8 内存管理核心结构职责

4.1.3 Boot 阶段的预映射

内核启动后会先初始化堆分配器和物理页帧分配器，再创建 `KERNEL_SPACE`。内核地址空间按照链接脚本导出的段边界分别映射 `.text`、`.rodata`、`.data`、`.bss` 与初始栈，并把 `ekernel` 之后到物理内存结束的区域作为可读写线性映射加入页表。设备 MMIO 区域同样以 `MapType::Direct` 方式映射到内核高地址，供 `VirtIO` 等驱动访问。

这部分映射必须在用户任务创建之前完成，因为用户页表的内核高地址部分会从 `KERNEL_SPACE` 派生。如果内核段权限配置错误，结果通常不是单个用户进程崩溃，而是 `trap`、调度、文件系统或设备访问路径整体不可用；因此内核段按照最小权限区分可执行、只读和可写区域。

需要注意的是，线性映射和页帧分配不是同一件事。线性映射只是让内核能够通过 `KERNEL_BASE + pa` 访问这段物理内存；某个物理页是否已经被占用，仍然由页帧分配器及其 `FrameTracker` 所有权决定。因此 `ekernel` 之后的物理区间既可以被高地址线性访问，也可以作为空闲页帧池的一部分被逐页分配。

4.1.4 内核地址转换

物理页内容通过 `PhysPageNum::get_bytes_array`、`get_pte_array` 和 `get_mut` 访问。RISC-V 路径始终将物理地址加上 `KERNEL_BASE` 得到内核虚拟地址；LoongArch 路径在分页开启后采用同样的高地址线性映射，分页开启前则保留直接物理访问能力。这样页表页、用户页和文件页都可以通过统一的物理页接口读写，而不需要临时把用户虚拟页映射到内核地址空间。

4.2 物理内存管理

4.2.1 物理页帧分配器

物理页帧分配器管理从内核镜像结束到 `MEMORY_END` 之间的可用物理页。初始化时，内核根据链接符号 `ekernel` 计算可分配起点，并与平台内存起点取较大值，避免把内核镜像、启动栈或已占用区域重新纳入页帧池。

当前实现采用栈式分配器：从连续页帧区间顺序分配新页，释放后的页号进入 `recycled` 栈，后续优先复用。这个策略实现简单，适合初赛阶段以正确性和可调试性为主的需求；页帧不足时返回 `None`，上层再转换为 `ENOMEM` 或相应错误路径。

4.2.2 物理页帧的生命周期管理

`frame_alloc` 返回 `FrameTracker`，构造时会清零整页，释放时通过 `Drop` 自动归还页帧。用户数据页和页表页都由 `FrameTracker` 持有，区别在于前者通常被 `MapArea.data_frames` 记录，后者由 `PageTable.frames` 记录。这样地址空间销毁、逻辑段拆分、`munmap` 或 `COW` 换页时，只要对应容器移除所有权，物理页就会自动进入回收路径。

三级页表本身也会消耗物理页：根页表至少占用一页，每遇到新的中间级索引还会再分配页表页。实际开销取决于虚拟地址分布，连续小进程通常只需要少量页表页，而 `mmap`、用户栈、动态链接器和高地址 `trampoline` 分散到不同区间时，会额外增加中间级页表页数量。

场景	持有者	释放条件
页表页	<code>PageTable.frames</code>	地址空间销毁或 LoongArch 页表延迟退休后释放。
普通用户页	<code>MapArea.data_frames</code>	逻辑段 <code>unmap</code> 、地址空间销毁或 COW 换页后引用归零。
共享映射页	<code>Arc<FrameTracker></code>	所有共享地址空间都解除映射后释放。

表 9 页帧生命周期入口

4.3 进程内存管理

4.3.1 虚拟内存区域

`MapArea` 是进程虚拟内存区域的基本单位。它不只记录虚拟页范围，还记录映射类型、权限、是否共享和已分配页帧集合。立即映射的区域通过 `push_empty_map_area` 建立全部页表项；懒分配区域通过 `push_map_area_lazy` 只记录范围，真正的物理页在首次访问缺页时分配。

为了支持 `munmap`、`mprotect` 和 `mmap` 覆盖语义，`MapArea` 可以按重叠区间拆分成左、中、右三段，并把对应的 `data_frames` 一起切分。这个设计避免了“一个 VMA 只能整体删除”的限制，也让部分解除映射后仍能保留剩余区间的权限和共享属性。

4.3.2 进程地址空间

创建新用户进程时，`from_elf_data` 解析 ELF 程序头，将每个 LOAD 段映射为用户 `MapArea`，再根据需要加载动态链接器并构造 `auxv`。用户栈、guard page、堆起点和 `mmap` 区间的相对位置已经在地址空间布局小节说明；本节关注的是这些区域如何由 `MemorySet` 记录并在 `execve`、`fork`、`brk`、`mmap` 路径中演进。

`fork` 路径通过 `from_existed_user` 创建子地址空间。只读页可以直接共享；可写页在父子两边都移除写权限并设置 COW 标志；尚未实际分配的懒分配页只复制虚拟范围，不复制物理页。这样普通 `fork` 不需要立即复制整个地址空间，只有父进程或子进程首次写共享页时才触发真正的数据复制。

4.4 缺页异常处理

4.4.1 进程缺页异常概述

用户态缺页由第三章的 `trap` 层识别后转交给 `MemorySet::handle_page_fault`。内存管理层先根据 `fault` 地址定位所属 `MapArea`，再检查该区域是否为用户映射、访问类型是否满足权限，最后根据页表项状态选择 COW 修复、懒分配或报错。缺页失败会返回错误，由 `trap` 层转化为 `SIGSEGV` 或任务退出。

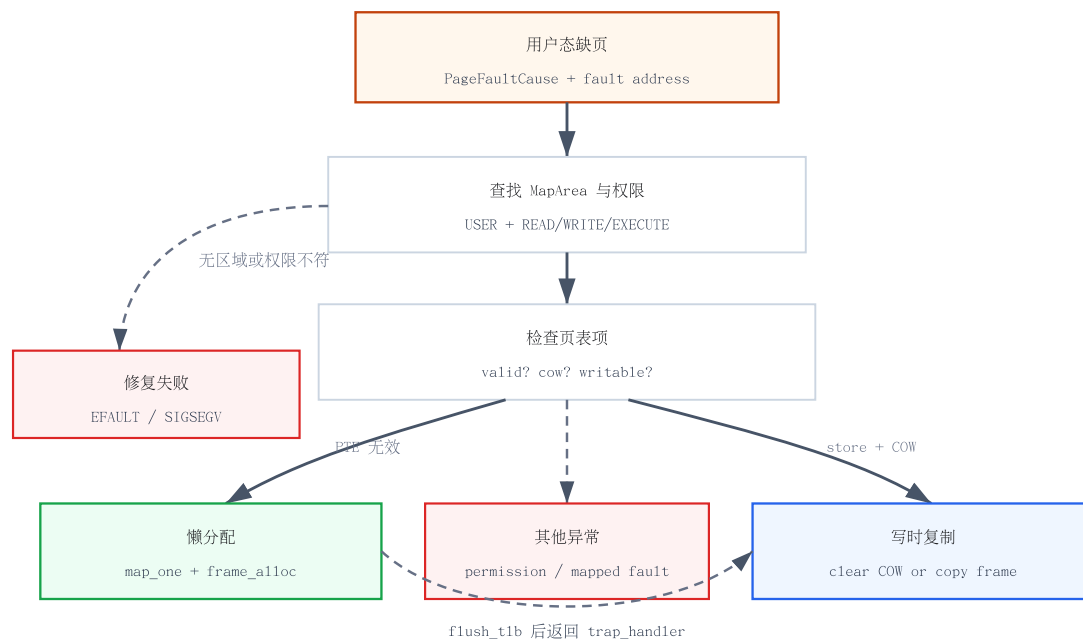


图 9 缺页异常处理流程

4.4.2 写时复制机制

写时复制只在写访问触发、页表项有效且带 COW 标志、所属 MapArea 允许写入时生效。若共享物理页的 Arc 引用计数为 1，说明已经没有其他地址空间共享该页，内核只需要恢复页表写权限并清除 COW 标志；若引用计数大于 1，则分配新页帧、复制旧页数据、替换当前地址空间的页表项和 data_frames 记录。

```

if is_store && pte.is_some_and(|p| p.is_valid() && p.is_cow()) {
    let old_frame = area.data_frames.get(&vpn).ok_or(Errno::EFAULT)?;
    if Arc::strong_count(old_frame) == 1 {
        page_table.modify_pte(vpn, PTEFlags::from(area_perm));
        page_table.clear_pte_cow(vpn);
    } else {
        area.remap_one_with_data(&mut page_table, vpn, old_frame.ppn().get_bytes_array()?);
    }
}

```

完成 COW 修复后必须刷新 TLB，否则硬件可能继续使用旧的只读页表项，导致同一地址反复触发写缺页。COW 的关键不是“延迟复制”本身，而是父子地址空间的页表权限、软件 COW 标志和物理页引用计数三者必须同步变化。

4.4.3 懒分配机制

懒分配用于匿名 mmap、堆扩展以及尚未触碰的用户区域。若 fault 地址位于合法 MapArea 内，但页表项不存在或无效，内核会根据 fault 类型推导所需权限：写访问需要 WRITE，极少情况下取指缺页需要 EXECUTE，读访问需要 READ。权限满足时，map_one 分配新物理页、写入页表项，并把页帧记录到 data_frames。

这种机制减少了大块匿名映射和堆扩展的初始成本，也避免 fork 时复制从未触碰过的页面。代价是用户指针访问不能简单地直接解引用用户地址，因为目标页可能尚未建立物理映射；这也是用户地址检查和 copy 接口需要主动触发缺页修复的原因。

4.5 内核动态内存分配

内核堆使用 `buddy_system_allocator::LockedHeap`，堆空间静态存放在 `.bss` 段中的 `HEAP_SPACE`，大小由 `KERNEL_HEAP_SIZE` 控制，目前为 64 MiB。初始化时，`init_heap` 将这段连续内存交给全局分配器，后续 `Arc`、`Vec`、`BTreeMap`、路径字符串、文件系统缓存元数据和任务结构都依赖该堆分配能力。

内核堆和物理页帧分配器职责不同。堆分配器面向内核对象和小粒度动态内存，页帧分配器面向页表页、用户数据页和大块页级映射。两者都可能因资源不足失败，但错误处理层次不同：堆分配失败会触发 `allocator panic`，页帧不足通常通过 `frame_alloc` 返回 `None` 并向系统调用路径报告 `ENOMEM`。

4.6 用户地址检查

用户态指针进入内核后，RespOS 不直接解引用用户虚拟地址，而是先通过 `check_user_readable` 或 `check_user_writable` 验证范围属于用户 `MapArea` 且权限满足要求，再调用 `ensure_user_page_access` 主动处理懒分配或 COW。实际拷贝时，内核逐页查页表，把用户虚拟地址翻译到物理页帧，再通过 `get_bytes_array` 完成跨页复制。

这种设计把“地址是否合法”和“页是否已经实际分配”分开处理。前者由 `MapArea` 权限判断保证，后者由缺页修复逻辑保证；如果任一阶段失败，系统调用返回 `EFAULT`。对于 `read`、`write`、`stat`、`poll`、`mmap` 等大量使用用户缓冲区的接口，这比在内核态触发 `page fault` 后再兜底更可控，也更容易定位 ABI 兼容问题。

本章小结

内存管理章的核心是把地址空间布局、页表元数据、物理页所有权和缺页修复分层。线性映射解决内核访问物理页的问题，页帧分配器决定页是否被占用，`MemorySet` 和 `MapArea` 则维护用户进程看到的虚拟内存语义。

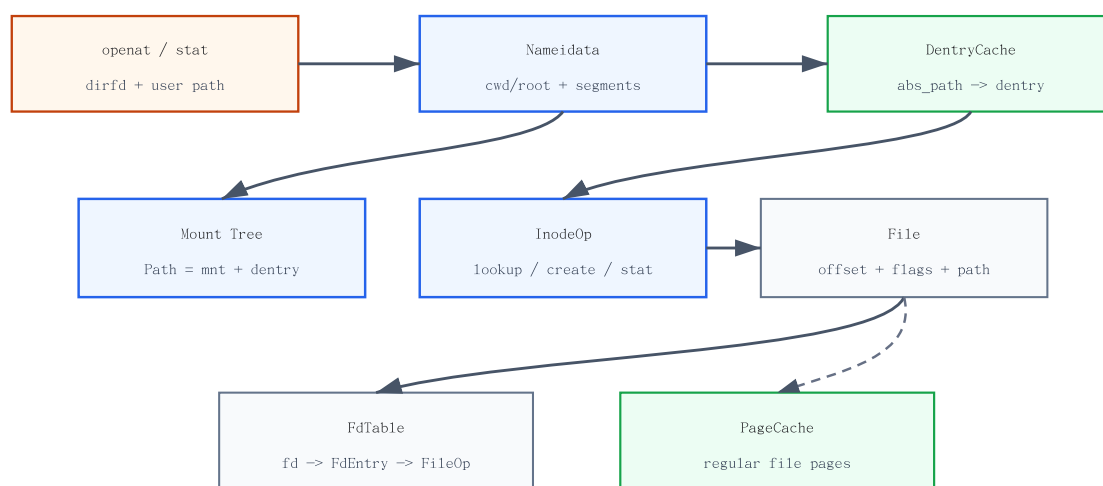
第五章

文件系统

5.1 虚拟文件系统

文件系统模块为用户态提供 Linux 风格路径、文件描述符、目录项和挂载语义。RespOS 没有把系统调用直接绑定到某一种磁盘格式，而是在 `fs/vfs/` 中定义 `SuperBlockOp`、`InodeOp`、`Dentry` 和 `FileOp` 等抽象，再由 `Ext4`、`procfs`、`devfs`、`pipe`、`stdio` 和特殊 `fd` 对象分别实现对应能力。

VFS 的核心目标是把“路径解析”“文件对象语义”和“打开文件状态”分开。路径解析得到的是 `Path { mnt, dentry }`，表示某个挂载实例中的目录项；`InodeOp` 描述该对象支持的元数据和读写接口；`File` 保存一次打开后的偏移量、打开标志和页缓存引用；`FdTable` 再把用户可见的整数 `fd` 映射到 `FileOp` 对象。



路径解析负责“找到对象”，打开文件负责“保存一次打开状态”，`fd` 表负责“把整数句柄映射到 `FileOp`”。

图 10 VFS 路径解析与打开文件流程

5.1.1 SuperBlock

`SuperBlockOp` 表示一个文件系统实例，至少需要提供根 `inode` 和同步接口，并可选实现 `statfs`。`Ext4`、`procfs` 和 `devfs` 都通过不同的 `super block` 挂载到同一棵 `mount tree` 中，因此系统调用层可以统一调用 `Path.mnt.fs.statfs()` 获取文件系统统计信息，而不需要知道路径最终落在哪个后端。

根文件系统在 `init_root_fs` 中建立：内核先通过 `fs/ext4` 创建 `Ext4 super block` 和根 `inode`，把它作为 `/` 的 `root mount` 加入挂载树，然后再把 `procfs` 和 `devfs` 分别挂载到 `/proc` 与 `/dev`。

挂载树由 `VfsMount` 和 `Mount` 维护。`VfsMount` 保存一个文件系统实例的根目录项和 `superblock`, `Mount` 记录它挂载在父文件系统的哪个 `dentry` 上。路径解析遇到挂载点时会切换到子挂载实例的根 `dentry`; 处理 `..` 时如果已经位于当前挂载根, 还需要回到父挂载点, 避免路径穿越逻辑与普通目录父子关系混在一起。

5.1.2 Inode

`InodeOp` 表示文件系统对象本身, 负责 `stat`、`read_at`、`write_at`、`truncate`、`lookup`、`readdir`、`create`、`link`、`unlink` 等语义。它不保存“这次打开文件的偏移量”, 也不直接暴露 `fd`; 这些状态属于 `File` 和 `FdTable`。这种拆分使同一个 `inode` 可以被多个文件描述符共享, 同时每个打开文件仍有独立 `offset` 和 `flags`。

`Ext4 inode` 会将底层 `lwext4` 错误码映射为 Linux `errno`, 并通过 `inode cache` 复用同一 `inode` 对象; 常规文件 `inode` 还持有共享 `PageCache`。`procfs` 和 `devfs` 的 `inode` 则通常不对应磁盘块, 而是根据当前任务、系统状态或设备对象动态生成目录项和文件内容。

5.1.3 Dentry

`Dentry` 是路径名缓存和目录树关系的承载对象, 保存绝对路径、父目录、弱引用子目录项和可选 `inode`。路径解析时, `Nameidata` 从当前任务的 `cwd` 或 `root` 出发, 按路径分量逐级查找; 先查全局 `dentry cache`, 未命中时调用当前目录 `inode` 的 `lookup`, 再把新 `dentry` 插入父目录和缓存。

`dentry` 和 `inode` 的职责不同: `dentry` 关心“某个名字在目录树中的位置”, `inode` 关心“该对象支持哪些文件操作”。硬链接、挂载点和符号链接都会让名字关系变复杂, 因此 VFS 需要 `dentry` 层来保存路径结构, 而不能只靠 `inode` 号描述用户看到的文件树。

5.1.4 File

`File` 表示一次打开文件, 内部保存 `inode`、当前 `offset`、打开标志、所属 `Path` 和可选页缓存。`read`、`write` 会根据 `offset` 调用页缓存或底层 `inode`, 并在成功后推进 `offset`; `O_APPEND` 写入前会把 `offset` 调整到当前文件末尾; `O_TRUNC` 在打开时截断常规文件并同步调整页缓存长度。

`File` 实现了 `FileOp trait`, `fd` 表中存储的是 `Arc<dyn FileOp>`, 通过 `trait object` 统一分派到常规文件、管道、设备等后端。这样 `read`、`write`、`poll`、`fcntl` 等系统调用只需要先通过 `fd` 找到 `FileOp`, 再按对象能力分派, 不需要在系统调用入口硬编码所有后端类型。

5.2 磁盘文件系统

5.2.1 EXT4 文件系统

RespOS 的磁盘文件系统后端基于 `lwext4_rust`。初始化时, `fs/ext4/mod.rs` 获取 `VirtIO block` 设备并交给 `Disk` 适配层; `Disk` 再把块设备的读、写和刷新能力转换成 `lwext4` 所需的 `KernelDevOp` 连续读写接口。

`Ext4SuperBlock::new` 创建 `Ext4BlockWrapper<Disk>` 时会完成 `lwext4` 挂载和 `superblock` 读取, 随后构造 `inode 2` 作为根 `inode`。`Ext4 inode` 通过 `lwext4 bindings` 执行目录查找、文件读写、创建、链接、重命名、符号链接和 `statfs` 等操作; 底层错误码由 `Ext4Inode::map_lwext4_err` 转成 `ENOENT`、`EIO`、`EEXIST`、`ENOTDIR`、`ENOSPC` 等内核 `Errno`, 未知错误统一按 `EIO` 处理。

为了减少重复构造 `inode` 对象, `Ext4` 层维护 `EXT4_INODE_CACHE`, 以 `inode` 号映射到弱引用。常规文件的 `Ext4Inode` 还关联一个共享 `PageCache`, 同一 `inode` 的多个 `File` 打开实例可以共享缓

存页，但每个 File 仍保留自己的 offset 和打开标志。对 `lwext4` 的修改操作通过 `EXT4_OP_LOCK` 串行化，避免底层 C 库状态在并发路径中被破坏。

5.3 非磁盘文件系统

5.3.1 procfs

`procfs` 用于向用户态暴露内核和进程状态。初始化时，内核在根文件系统中创建 `/proc` 挂载点，再挂载 `ProcSuperBlock`；`/proc/self`、`/proc/self/fd`、`/proc/self/maps`、`/proc/self/smaps`、`/proc/self/stat`、`/proc/cpuinfo`、`/proc/meminfo` 等文件由对应虚拟 inode 动态生成内容。

`procfs` 的文件多数没有持久化数据块。读取时 inode 根据当前任务、内存映射、fd 表或系统统计信息生成文本；写入、链接和删除通常返回只读或不支持错误。这个后端主要服务 `libc`、`busybox` 和 `LTP` 对 Linux `/proc` 兼容性的依赖。

5.3.2 devfs

`devfs` 在 `/dev` 下提供设备和特殊文件入口，包括 `/dev/null`、`/dev/zero`、`/dev/random`、`/dev/urandom`、`/dev/shm`、`/dev/rtc`、`/dev/loop-control` 和 `loop0` 等。它同样通过 VFS 挂载树接入，但具体 inode 的读写语义由设备类型决定，例如 `/dev/null` 读取返回 EOF、写入丢弃数据，`/dev/zero` 读取填充零字节。

`devfs` 的意义不只是提供几个路径名，而是把设备对象纳入统一的 fd 和路径模型。用户程序可以用普通 `openat`、`read`、`write`、`stat` 访问设备文件，系统调用层无需为每个设备额外设计入口。

5.4 页缓存

页缓存位于 VFS 与磁盘后端之间，按 inode 共享常规文件的数据页。`PageCache` 以页号索引缓存页，每页包含 4 KiB 数据、dirty 标志、LRU generation 和队列状态。读取未命中时，页缓存会在锁外调用底层 inode 的 `read_at` 加载数据；写入时先修改缓存页并标记 dirty，`fsync` 或 `File drop` 时再通过 `sync` 写回底层文件。

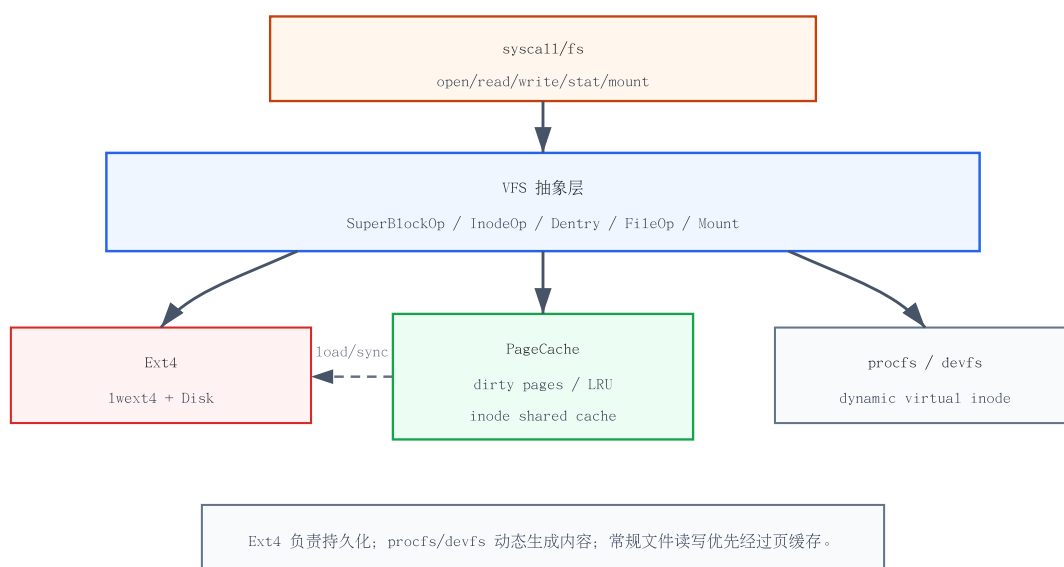


图 11 文件系统后端与页缓存关系

全局页缓存通过 `PAGE_CACHE_LRU` 和 `PAGE_CACHE_PAGE_COUNT` 控制规模。只有未 `dirty`、`generation` 未变化且没有额外强引用的页才能被回收；`dirty` 页需要先写回，仍被文件或其他路径持有的页也不能直接丢弃。这个策略避免了简单粗暴释放缓存导致的数据丢失，同时让常规文件重复读写可以绕开底层磁盘 I/O。

5.5 文件描述符表

`FdTable` 是进程资源的一部分，保存 `Vec<Option<FdEntry>>`、下一个可用 `fd` 和 `RLIMIT_NOFILE` 边界；它也对第二章 TCB 中的进程资源字段。新任务创建时默认拥有 0、1、2 三个标准 `fd`；`openat` 分配最小可用 `fd`，`dup/fcntl` 可以从指定下界查找空位，`close` 会释放表项并回退 `next_fd`。`execve` 前会根据 `O_CLOEXEC` 清理需要关闭的描述符。

`FdEntry` 保存 `Arc<dyn FileOp>` 和 `fd flags`。`fork` 时 `fd` 表会复制表项，但底层 `FileOp` 通过 `Arc` 共享，因此父子进程可以共享同一个打开文件对象和 `offset`；这符合大量 Unix 程序对 `fork` 后文件偏移共享的预期。对于 `pipe`、`socket`、`eventfd`、`timerfd` 等特殊对象，`fd` 表不需要知道内部结构，只负责持有和查找 `FileOp`。

本章小结

文件系统章的核心设计是把路径、挂载、`inode`、打开文件和 `fd` 表拆开：`VFS` 保持统一语义，`Ext4` 负责持久化，`procfs/devfs` 负责兼容性入口，页缓存承担常规文件的性能边界。这样的分层让后续增加 `pipe`、`socket`、`eventfd` 等对象时可以复用 `fd` 模型，而不需要反复修改系统调用入口。

第六章

进程间通信

6.1 信号机制

信号机制用于把异步事件递送给用户任务，是进程控制、异常通知、定时器和阻塞系统调用中断的共同基础。RespOS 将信号状态拆成三部分：**SigPending** 保存未决信号位图、当前屏蔽集和 **SigInfo**；**SigHandler** 保存每个信号的处理动作；任务控制块负责在阻塞、停止、继续和退出路径中与调度器协作。

信号不会在发送时立即改写用户上下文。发送路径只负责构造 **SigInfo** 并放入目标任务的 pending 集合；真正递送发生在第三章所述的 trap 返回用户态之前，由 **handle_signal** 检查未被屏蔽的未决信号。这样可以保证内核总是在一个明确的边界修改 **TrapContext**，避免在任意内核执行点强行插入用户 handler。

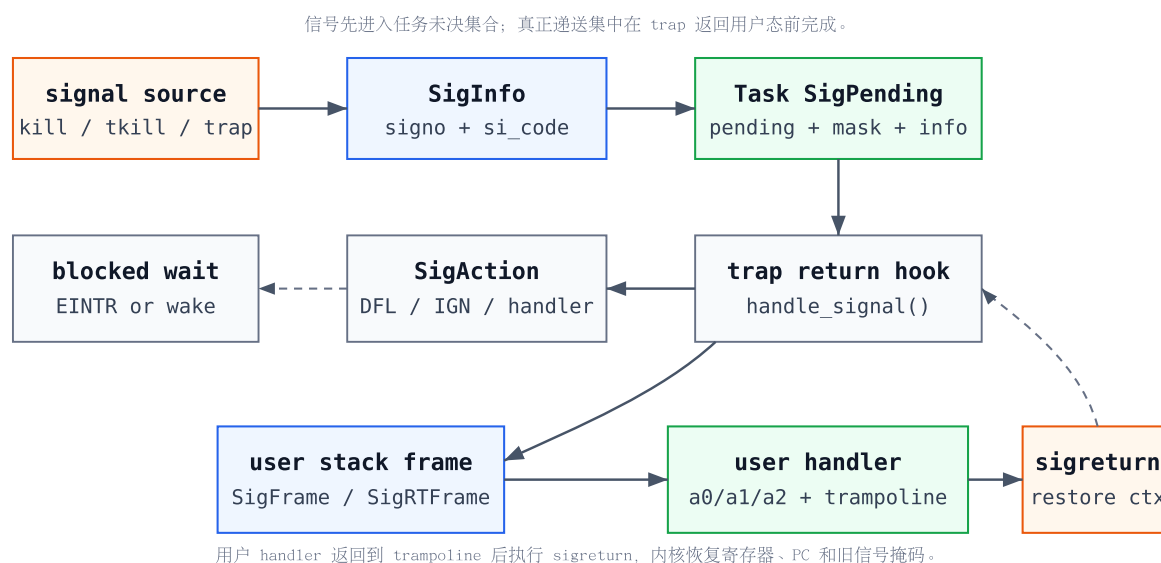


图 12 信号递送与返回流程

6.2 信号传输

信号来源主要包括三类。第一类是用户显式发送，**kill** 按 PID、进程组或全体任务选择目标，**tkill** 按 TID 发送，**tgkill** 同时校验 TGID 与 TID，避免把信号误送给已经复用的线程号。第二类是内核异常生成，例如非法指令导致任务终止、缺页失败递送 **SIGSEGV**、管道破裂递送 **SIGPIPE**。第三类来自时间和等待路径，例如 **real timer** 到期、阻塞系统调用被信号打断。

传输阶段还需要处理 Linux 权限和线程语义。**kill** 会检查发送者和目标任务的 **uid**、**euid**、**suid**，非特权任务不能随意向其他用户进程发送信号；**SIGKILL** 和 **SIGSTOP** 不能被屏蔽；**SIGCONT** 到达停止任务时会更新 **wait** 事件并唤醒任务。面向线程的信号则进入具体 TID 的 pending 集合，面向进程的信号可以根据线程组选择可接收的线程。

接口	职责
<code>kill</code>	按 PID、进程组或全体任务选择目标，并执行权限检查。
<code>tkill / tgkill</code>	向具体线程发送信号，其中 <code>tgkill</code> 额外校验线程组。
<code>rt_sigaction</code>	安装或查询用户 <code>handler</code> ，禁止修改 <code>SIGKILL</code> 与 <code>SIGSTOP</code> 。
<code>rt_sigprocmask</code>	修改当前任务屏蔽集，并自动移除不可屏蔽信号。
<code>rt_sigsuspend</code>	临时替换屏蔽集并进入可中断等待，醒来后返回 <code>EINTR</code> 。
<code>rt_sigtimedwait</code>	从指定集合中消费 <code>pending</code> 信号，可带超时。
<code>sigaltstack</code>	设置备用信号栈，供 <code>SA_ONSTACK</code> <code>handler</code> 使用。
<code>sigreturn</code>	从用户栈读取信号帧，恢复寄存器、PC 和旧屏蔽集。

表 10 信号相关系统调用职责

6.3 信号处理

信号处理阶段由 `SigAction` 决定最终动作。若 `handler` 为 `SIG_IGN`，内核直接忽略；若为 `SIG_DFL`，根据默认动作执行终止、停止、继续或忽略；若为用户函数地址，内核会在用户栈或备用信号栈上构造信号帧，并把 `TrapContext` 的返回地址改写到用户 `handler`。

RespOS 支持普通信号帧和 `SA_SIGINFO` 实时信号帧。普通帧保存 `SigContext`，记录被打断时的通用寄存器、返回 PC 和旧屏蔽集；`SA_SIGINFO` 路径额外压入 `LinuxSigInfo` 和 `UContext`，并把 `handler` 参数设置为 `signo`、`siginfo_t*`、`ucontext_t*`。栈帧开头带 `FrameFlags`，`sigreturn` 依靠它区分普通帧和 RT 帧。

标志	效果
<code>SA_SIGINFO</code>	使用 RT 信号帧，向 <code>handler</code> 传入 <code>siginfo_t*</code> 和 <code>ucontext_t*</code> 。
<code>SA_NODEFER</code>	<code>handler</code> 执行时不自动屏蔽当前信号。
<code>SA_RESETHAND</code>	信号递送后把该信号动作恢复为默认值。
<code>SA_ONSTACK</code>	在备用信号栈上执行 <code>handler</code> 。

表 11 信号处理相关 SA 标志

用户 `handler` 返回到架构相关的 `trampoline` 后进入 `sys_sigreturn`，内核再从用户栈读回保存的上下文，恢复 `TrapContext` 和信号屏蔽集。

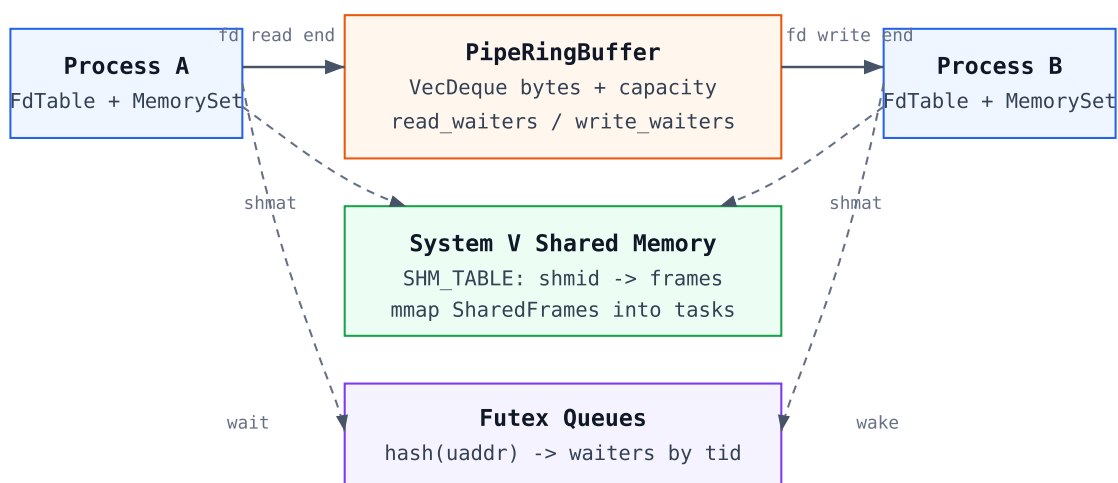
6.4 管道 (Pipe)

6.4.1 Pipe 设计

管道是通过文件描述符暴露的字节流 IPC。`pipe2` 创建一个共享 `PipeRingBuffer`，并返回读端和写端两个 `Pipe` 对象；二者都实现 `FileOp`，复用第五章介绍的 `trait object` 分派模型，因此可以被 `read`、`write`、`poll`、`fcntl`、`close` 和 `splice` 等文件系统路径统一处理。管道不可 `seek`，相关接口返回 `ESPIPE`。

PipeRingBuffer 使用 VecDeque<u8> 保存数据，并维护容量、读端关闭标志、写端关闭标志、读等待队列和写等待队列。默认容量受 PIPE_BUFFER_SIZE 和 /proc/sys/fs/pipe-max-size 影响，F_GETPIPE_SZ / F_SETPIPE_SZ 可以查询或调整容量；调小容量时若小于当前已缓存字节数，会返回 EBUSY。

```
struct PipeRingBuffer {
    buffer: VecDeque<u8>,
    capacity: usize,
    read_closed: bool,
    write_closed: bool,
    read_waiters: VecDeque<usize>,
    write_waiters: VecDeque<usize>,
}
```



管道通过 fd 传递字节流；共享内存通过同一批页帧共享数据；futex 只在用户地址上建立等待/唤醒关系。

图 13 IPC 对象与任务关系

6.4.2 读端与写端通信

读管道时，如果缓冲区已有数据，内核直接拷贝并唤醒一个写等待者；如果缓冲区为空且写端已经关闭，则返回 0 表示 EOF；如果缓冲区为空但写端仍存在，当前任务进入可中断阻塞状态，并把 TID 放入 read_waiters。写管道时，如果缓冲区有空间，内核写入数据并唤醒一个读等待者；如果缓冲区满，写端进入 write_waiters；如果读端已经关闭，则返回 EPIPE。

管道阻塞路径必须和信号机制配合。任务进入等待队列前会设置 interruptible 标志，醒来后检查 is_interrupted 和 check_signal_interrupt，若被信号打断则返回 EINTR 并从等待队列移除自身。这样 read、write、poll、splice 等接口既能等待对端，又不会吞掉用户期望观察到的信号。

6.5 System V IPC 机制

System V IPC 当前主要覆盖共享内存子集，目标是满足 libc、busybox 和 LTP 中常见的 shmget、shmat、shmctl、shmdt 路径。它与管道的区别在于：管道传递的是内核缓冲区中的字节流，共享内存则把同一批物理页映射到多个进程地址空间中，读写动作由用户态直接完成。

6.5.1 System V IPC 对象

全局 `SHM_TABLE` 保存所有共享内存段，表项由 `shmid` 索引。`ShmSegment` 是表中的共享内存对象，保存段大小和一组 `Arc<FrameTracker>`；`shmget` 会按页对齐请求长度，分配对应数量的物理页帧，并把这些页帧放入新的 `ShmSegment`。这些页帧不属于某个单独进程，而是由共享内存对象持有，进程只是在自己的地址空间中引用它们。

`shmat` 从 `SHM_TABLE` 查出段对象后，把其中的页帧作为 `MmapBacking::SharedFrames` 映射进当前任务的 `MemorySet`。未设置 `SHM_RDONLY` 时映射带 `READ | WRITE | USER` 权限，设置只读标志时去掉写权限。映射完成后刷新 TLB，使用户态立即看到新的地址空间状态。

6.5.2 IPC Key 管理器

目前实现对 `key` 语义采用保守子集：`IPC_PRIVATE` 总是创建新段；非私有 `key` 若没有 `IPC_CREAT` 则返回 `ENOENT`。全局表用递增的 `next_id` 分配 `shmid`，而不是维护完整 `key` 到对象的复用关系。这种做法可以覆盖初赛阶段大量“创建、附着、读写、删除”的共享内存用例，但还不是完整 System V IPC 命名空间实现。

完整 Linux 语义还需要处理 `key` 复用、权限位、`IPC_EXCL`、引用计数、`namespace` 和 `shmid_ds` 元数据。RespOS 当前把这些复杂度暂时压缩在 `syscall/ipc.rs` 中，后续若需要补齐 System V semaphore 或 message queue，可以继续沿用“全局表 + ID 分配 + 对象元数据”的组织方式。

6.5.3 System V 共享内存

共享内存的生命周期由四个系统调用串起来。`shmget` 创建段并返回 `shmid`；`shmat` 把段映射到当前进程，地址可以由内核选择，也可以由用户指定页对齐地址；`shmdt` 按起始地址移除对应 VMA 并刷新 TLB；`shmctl(shmid, IPC_RMID, NULL)` 用 `IPC_RMID` 命令从全局表删除段。由于实际数据页由 `Arc<FrameTracker>` 持有，已经映射到进程地址空间中的共享页会在引用归零后释放。

这种设计和普通匿名 `mmap` 的关键差异在于页帧来源。匿名映射通常在缺页时给每个进程分配自己的页；System V 共享内存则在段创建时分配一组共享页，多个进程的页表项指向同一批物理页。因此一个进程写入共享页后，其他已附着进程可以直接通过自己的虚拟地址观察到变化。

6.6 Futex 同步

`futex` 不是传统 System V IPC，但它是用户态线程库实现 `mutex`、`condvar` 和 `join` 语义的重要基础。RespOS 的 `do_futex` 在任务模块中解析 `FUTEX_WAIT`、`FUTEX_WAKE`、`FUTEX_REQUEUE`、`FUTEX_CMP_REQUEUE`、`FUTEX_WAIT_BITSET` 和 `FUTEX_WAKE_BITSET`，系统调用层只负责传入原始参数。

`futex` 等待队列按用户地址哈希到 256 个桶。私有 `futex` 的 `key` 包含线程组号和用户地址，共享 `futex` 当前使用全局 `scope`；等待前内核会先读取用户地址的实际值，若和期望值不一致则返回 `EAGAIN`，避免错过用户态已经完成的状态变化。等待任务同样以可中断方式阻塞，信号到达时返回 `EINTR`。

本章小结

进程间通信章围绕三类共享边界展开：信号共享的是控制流事件，管道共享的是内核字节缓冲，共享内存共享的是物理页帧，`futex` 则在用户地址上建立等待/唤醒关系。这些机制都必须和任务状态、信号中断、文件描述符表或地址空间管理协作，才能呈现接近 Linux 的用户态行为。

第七章

时钟模块

7.1 定时器队列

7.1.1 时间轮设计

7.2 定时器

第八章

网络模块

- 8.1 Socket 套接字
- 8.2 Ethernet 设备
- 8.3 传输层: UDP 与 TCP
- 8.4 Port 端口分配

第九章

设备

9.1 设备管理模块概述

9.2 设备树

第十章

硬件抽象层

- 10.1 硬件抽象层总览
- 10.2 处理器访问接口
- 10.3 内核入口例程
- 10.4 内存管理单元与地址空间
 - 10.4.1 物理内存
 - 10.4.2 分页地址翻译模式
 - 10.4.3 页表
 - 10.4.4 直接映射窗口
 - 10.4.5 TLB 重填

第十一章

总结与展望

11.1 工作总结

11.2 未来计划

11.3 参考

第十二章

AI 协作与开源项目借鉴

- 12.1 AI 辅助完成的工作
- 12.2 人工审核与边界控制
- 12.3 借鉴的开源项目
- 12.4 许可证与引用说明